

ai_assistant: assessment against industry standards

2026-09-26 · working tree at b524d2d plus uncommitted changes · cargo check passes with 46 warnings · not run end to end

Overall: about 5/10. The core design ("the model extracts, code decides") is sound and close to current best practice (it closely matches Rasa CALM). It is still a prototype: there are no tests or evals, no latency work, and nothing is wired up for production.

Requested categories

Category	Score	Why
Maintenance effort (higher = easier)	5	Helps: the code is small (about 2.5k lines of Rust). Exhaustive match guards and flat_enum! mean the compiler points you at every place to update. context.md is excellent. Hurts: no tests on the pure logic. There are 46 warnings and a lot of dead code (session.rs, routes/middleware.rs, vllm.rs.old). Settings and comments still describe the removed prompt-file design. Prompts live in Rust strings, so changing a prompt means a recompile. Commit messages ("save 1231213...") are too vague to trace changes.
Extensibility	6	Adding a form is one arm in FormSupported::build(). Adding an information source is a variant, a Display arm and a fetch arm. Machine/ValueSchema/OutputFormat is a good generic way to add LLM stages. Limits: forms are code, not data. Field kinds cover only primitives (no choice, time, phone or email). No validation beyond the type, no required flag, no submit or availability hooks, and only one value per turn. Event handlers are synchronous, so I/O has to live in the route. Every new form or information source grows one flat label list, and a 7B model's classification accuracy drops as that list reaches the dozens.
Readability	7	Good names, comments that explain why, and a handler that reads as numbered steps 1 to 5. Against that: not rustfmt'd, unused imports, stale comments (classify, prompt hot-reload), machine.rs mixes the framework with both machines' definitions, and the mem::take state in and out around dispatch is a workaround readers have to decode.
Response speed	4	Good: the prompt order suits prefix caching (static, then history, then volatile), temperature is 0, max_tokens is capped, repeat skips the LLM, and inference runs locally. Bad: two sequential LLM calls, no streaming into TTS, and reasoning is the first output field, so it is generated before the intent. The whole unbounded history goes into both prompts, deterministic turns still go through the LLM, and the timeout is 120 s. Rough estimate (not measured): 1.5–3 s per turn before STT/TTS, against the usual target of under 1 s.
Flexibility	5	The OpenAI-compatible client and env config make swapping models easy. Prompts, forms, the city (Berlin) and the currency (EUR→UAH) are compiled in. Single tenant, with no per-deployment configuration of intents or forms.

Additional categories

Category	Score	Why
Architecture / design	8	The LLM extracts and deterministic code decides. Typed state machine, state rolled back when the formulator fails, turns of a call serialized by a lock. One deduction: the event bus adds indirection with no benefit yet (one handler, one big match).
Testability and LLM evaluation	2	The form logic and CallStateHandler are pure and easy to test, but none of them have tests. The existing tests need live Postgres and Redis, and two are <code>assert!(true)</code> . There's no eval set of utterances with expected intents, so a prompt change can't be checked for regressions.
Reliability	4	It uses <code>json_object</code> , not <code>json_schema</code> , so the model isn't forced to pick an allowed intent. An invalid answer becomes a 502 with no retry. Panic paths: <code>expect</code> in <code>Machine::query</code> , Postgres required at boot though unused, and <code>/test-session</code> panics. A corrupt Redis slot silently drops a form. Truncation detection was removed.
Security and privacy	3	Escaping tags in the prompt is good. On the other side: full prompts (names, dates of birth, visit reasons) are logged at <code>info</code> , and visit reasons are health data (GDPR Art. 9). <code>x-call-id</code> is trusted with no auth, CORS is <code>permissive()</code> , and a DB password is committed in <code>.env.dist</code> .
Scalability	4	The app is almost stateless (state in Redis). The call lock only works within one process (last write wins across instances), history in Redis grows without bound, and each slot is JSON inside <code>MessagePack</code> , so it's encoded twice.
Conversation design	6	Handles corrections, digressions, repeat, escalation after three rejections, and hand-off. But each field is confirmed on its own turn (four fields take at least eight turns), it takes one value per sentence, there's no handling for speech-recognition noise or "I don't know", and it doesn't record which question is pending.
Observability	5	Structured tracing with <code>call_id</code> and rolling files. No metrics (latency per stage, intent mix, completion rate) and no OpenTelemetry.
Production readiness	3	Completed forms go nowhere, there's no booking or availability backend, end call and transfer only set a flag, and it has never run end to end.
Documentation	8	<code>context.md</code> and the handbook are well above average. Parts are already stale (fixed middleware and header issues are still listed, and <code>prompts/README.md</code> describes the old design).

Highest-impact next steps

1. **Enforce the schema during generation.** Use vLLM `response_format: json_schema` with an enum for `caller_intent`, taken from `all_labels()`. Put the intent first, or drop the reasoning field.
2. **Tests and evals:** unit tests for `Form` and `CallStateHandler`, plus a set of 50–200 utterance/state/expected-intent cases run against the real model.
3. **Latency:** templates instead of machine #2 for deterministic turns, streaming output, a history window of 4–6 turns, and a timeout of about 5–8 s with a spoken fallback and one retry.
4. **Forms as data** (declarative fields, choice kinds, validators, required flag) with an `async validate / lookup_availability / submit` hook. Booking needs availability checks while the form is being filled, not only at the end.
5. **Intent scaling:** pick a category before the specific intent (or retrieve candidate intents), and only offer intents that make sense in the current state.
6. **Security:** log prompts at debug with values redacted, authenticate the telephony side, and move the call lock to Redis (`SET NX PX`).
7. **Cleanup:** remove dead code and stale settings, use Postgres for completed forms or drop it, and run `rustfmt` and `clippy -D warnings` in CI.